# Design Patterns in C++

Krystian Piękoś - Infotraining

# Contents

# 1. Foundations of Object-Oriented Programming

## 1.1 Four tenets of object-oriented programming

Object-oriented programming (OOP) is a programming paradigm that uses “objects” to design applications and computer programs.

It utilizes several techniques to design and develop software. The four major principles of object-oriented programming are:

1. **Abstraction**
    - Abstraction is the concept of hiding the complex implementation details and showing only the necessary features of the object. It helps to reduce programming complexity and effort.
2. **Encapsulation**
    - Encapsulation (aka *Information Hiding*) is the process of bundling the **data** (variables) and the **methods** (functions) that operate on the data into a single unit called a class. All internal details of the class implementation are hidden from the outside world, and only the necessary interface that allows to operate on an object is exposed to the user.
3. **Polymorphism**
    - It allows to perform a single action in different ways. In other words, polymorphism allows to define one interface and have multiple implementations. Objects that have the same interface can be used interchangeably.
4. **Code Reusability: Composition & Inheritance**

Composition and inheritance are two ways to achieve code reusability in object-oriented programming.

- **Composition** is a “has-a” relationship between two classes. It allows creating complex types by combining objects of other types.
- **Inheritance** is an “is-a” relationship between two classes. It allows creating a new class that inherits attributes and methods from an existing class.

## 1.2 Object - Interface - Class

### 1.2.1 Object

An object is a real-world entity that has a state and behavior. It can be a physical entity or a conceptual entity. An object has three main characteristics:

1. **State**: It represents the data (attributes) of an object.
2. **Behavior**: It represents the methods (functions) that operate on the object’s state.
3. **Identity**: It gives a unique name to an object and enables one object to interact with another object.

In object oriented software objects communicate with each other using methods.

### 1.2.2 Interface

An interface is a public contract that defines the methods that a class must implement. It specifies what a class must do but not how it does it. Interfaces are used to define the behavior of an object.

Interface is separated from the objects implementation. Many objects that have the same interface can vary in their implementation.

### 1.2.3 Class

Class defines the implementation for an object. It is a blueprint for creating objects. It defines the attributes and methods that an object will have.

Class allows to instantiate objects. An object is an instance of a class.

#### 1.2.3.1 Abstract Class

An abstract class is a class that cannot be instantiated.

It is used to define a common interface for all the subclasses. It allows to define **abstract methods** that must be implemented by the subclasses.

Abstract methods are methods that are declared in the abstract class but do not have an implementation. In C++ they are declared as **pure virtual functions**.

Class that contains at least one pure virtual function is called an abstract class.

Class that implements all the pure virtual functions of an abstract class is called a **concrete class** and can be instantiated.

The code below shows an example of an abstract class `Shape` that defines an interface that contains two virtual functions `move()` and `draw()`. This abstract class has partial implementation. It has members `x` and `y` that represents coordinates. Having these coordinates one can implement the `move()` function. The `draw()` function is a pure virtual function that must be implemented by the derived classes.

```
class Shape
{
    int x_, y_;
public:
    Shape(int x = 0, int y = 0) : x_{x}, y_{y}
    {}

    virtual ~Shape() = default;

    virtual void move(int dx, int dy)
    {
        x_ += dx;
        y_ += dy;
    }

    virtual void draw() const = 0;
};

class Rectangle : public Shape
{
    int width_, height_;
public:
    Rectangle(int x, int y, int width, int height) : Shape(x, y),
width_{width}, height_{height}
    {}

    void draw() const override
    {
        std::cout << "Drawing a rectangle at (" << x_ << ", " << y_ << ") with
width " << width_ << " and height " << height_ << std::endl;
    }
};
```

Inheritance from an abstract class with a partial implementation can lead to some issues. For instance, when we want to represent a position of triangle vertices in an array then the derived members `x` and `y` are obsolete. This leads to a waste of memory and can cause a confusion where the state of the object is stored.

#### 1.2.3.2 Extraction of the Pure Interface

In order to avoid dependency on the base class implementation one can extract “the pure interface” - `Shape`. The abstract class `ShapeBase` with a partial implementation derives from

the interface. Concrete classes can choose to derive from the interface avoiding unnecessary coupling or from the abstract class avoiding code redundancy.

```
class Shape
{
public:
    virtual ~Shape() = default;

    virtual void move(int dx, int dy)  = 0;
    virtual void draw() const = 0;
};

struct Point { int x, y; };

class ShapeBase : public Shape
{
    Point coord_;
public:
    ShapeBase(int x, int y) : coord_{x, y}
    {}

    void move(int dx, int dy)  override
    {
        coord_.x += dx;
        coord_.y += dy;
    }
};
```

Now concrete classes can derive from the pure interface:

```
class Triangle : public Shape
{
    Point vertices_[3];
public:
    void move(int dx, int dy) override
    {
        for (auto& vertex : vertices_)
        {
            vertex.x += dx;
            vertex.y += dy;
        }
    }

    void draw() const override
    {
        std::cout << "Drawing a triangle [" << vertices_[0] << ", " <<
vertices_[1] << vertices_[2] << "]" << std::endl;
    }
};
```

or from the abstract class with implementation of coordinates and move.

```
class Rectangle : public ShapeBase
{
public:
    Rectangle(int x, int y, int width, int height) : ShapeBase{x, y}
    {
    }

    void draw() const override
    {
        std::cout << "Drawing a rectangle at (" << coord_.x << ", " <<
coord_.y << ") "
                  << "with width " << width_ << " and height " << height_ <<
std::endl;
```

```
    }
};
```

## 1.3 Polymorphism

Polymorphism is a feature of object-oriented programming that allows to perform a single action in different ways. It allows to define one interface and have multiple implementations. Objects that have the same interface can be used interchangeably.

In C++ polymorphism can be achieved in many ways:

- using **virtual functions** and **inheritance** - it is so called **run-time polymorphism** or **dynamic polymorphism**,
- using **templates** - it is so called **compile-time polymorphism** or **static polymorphism**,
- using **polymorphic wrappers** - this is form of dynamic polymorphism and it is also called as **duck typing**.

### 1.3.1 Dynamic Polymorphism

Dynamic polymorphism in C++ is achieved by using **virtual functions** and **inheritance**. It allows to define a common interface for a group of classes. Virtual functions from a base class can be overridden by the derived classes.

When a pointer or reference to a base class is used to refer to an object of derived class, the function call is resolved at runtime (via **virtual-table**). It allows to change the behavior required by some client at runtime.

Example:

```
class Formatter
{
public:
    virtual std::string format(const std::string& data) = 0;
    virtual ~Formatter() = default;
};

class Logger
{
    std::unique_ptr<Formatter> formatter_;

public:
    Logger(std::unique_ptr<Formatter> formatter)
        : formatter_{std::move(formatter)}
    { }

    void log(const std::string& data)
    {
        std::cout << "LOG: " << formatter_->format(data) << '\n';
    }
};
```

`Formatter` is an interface required by the `Logger` class. It defines a method `format()` that must be implemented by the derived classes. The `Logger` class uses the `Formatter` interface to format the data before logging it.

Now we can define concrete classes that implement the `Formatter` interface:

```
class UpperCaseFormatter : public Formatter {
public:
    std::string format(const std::string& data) override {
        std::string transformed_data{data};
        std::transform(data.begin(), data.end(), transformed_data.begin(),
            [](char c) { return std::toupper(c); });
        return transformed_data;
    }
```

```
};

class LowerCaseFormatter : public Formatter {
public:
    std::string format(const std::string& data) override {
        std::string transformed_data{data};
        std::transform(data.begin(), data.end(), transformed_data.begin(),
            [](char c) { return std::tolower(c); });
        return transformed_data;
    }
};
```

Now we can use the `Logger` class with different formatters:

```
Logger logger{std::make_unique<UpperCaseFormatter>()};
logger.log("Hello, World!");

logger = Logger{std::make_unique<LowerCaseFormatter>()};
logger.log("Hello, World!");
```

Polymorphism in this scenario allows to change the behavior of the `Logger` class at runtime. The `Logger` class is not dependent on the concrete implementation of the `Formatter` class. It uses the `Formatter` interface to format the data.

#### 1.3.1.1 Dynamic Polymorphism - Pros and Cons

**Pros:**

- **Flexibility**: Dynamic polymorphism allows to change the behavior of the object at runtime. It allows to define a common interface for a group of classes and use them interchangeably.
- **Loose Coupling**: Dynamic polymorphism allows to create a loose coupling between the base class and the derived classes. It allows to change the behavior of the base class without affecting the derived classes.

**Cons:**

- **Performance Overhead**: Dynamic polymorphism has a performance overhead. It requires to resolve the function calls at runtime. It uses **virtual-table** to resolve the function calls.
- **Memory Overhead**: Dynamic polymorphism requires to store additional information about the virtual functions. It requires to store a pointer to the virtual-table in each object.
- **Reference Semantics**: Dynamic polymorphism requires to use pointers or references to the base class. It can lead to a more complex code than code that uses *value semantics*. Often we need to dynamically allocate memory for the objects and use smart pointers to manage their lifetimes.
- **Requires Inheritance**: Dynamic polymorphism requires to use inheritance. It creates strong relationship between types. It can lead to a more complex design.

### 1.3.2 Static Polymorphism

Static polymorphism in C++ is achieved by using **templates**. When we pass a type with an interface required by the client as a template parameter, the client can use the type as if it was a polymorphic object. It allows to set the behavior of the object at compile time.

Example of a template class that uses a type that provides a `format()` method as a template parameter:

```
template <typename TFormatter = UpperCaseFormatter>
class Logger
{
    TFormatter formatter_;

public:
```

```
    Logger() = default;

    Logger(TFormatter formatter)
        : formatter_(std::move(formatter))
    {
    }

    void log(const std::string& message)
    {
        std::cout << formatter_.format(message) << std::endl;
    }
};
```

Now we can define concrete classes that provide the `format()` method:

```
struct UpperCaseFormatter
{
    std::string format(const std::string& message) const
    {
        std::string result = message;
        std::transform(result.begin(), result.end(),
            result.begin(), [](char c) { return std::toupper(c); });
        return result;
    }
};

struct CapitalizeFormatter
{
    std::string format(const std::string& message) const
    {
        std::string result = message;
        result[0] = std::toupper(result[0]);
        return result;
    }
};
```

Notice that both classes provide the `format()` method which is not virtual. There is no inheritance from a common base class. The `Logger` class uses the `format()` method provided by the template parameter.

Now we can parametrize the `Logger` class with different formatters at compile time:

```
Logger logger_1{UpperCaseFormatter{}};
logger_1.log("Hello, World!");

Logger<CapitalizeFormatter> logger2;
logger2.log("hello, world!");
```

**Tip**

In C++20 static polymorphism can be even more powerful with the introduction of **concepts**. Concepts allow to define constraints on template parameters. We can define concept that requires a type to provide a specific interface. It will allow to formalize the interfaces that is required by clients and provide better error messages when the requirements are not met.

```
template <typename T>
concept Formatter = requires(T formatter, const std::string& message)
{
    { formatter.format(message) } -> std::same_as<std::string>;
};

template <Formatter TFormatter>
class Logger
{
```

```
    TFormatter formatter_;

public:
    Logger() = default;

    Logger(TFormatter formatter)
        : formatter_(std::move(formatter))
    {
    }

    void log(const std::string& message)
    {
        std::cout << formatter_.format(message) << std::endl;
    }
};
```

#### 1.3.2.1 Static Polymorphism - Pros and Cons

**Pros:**

- **Performance**: Static polymorphism has no performance overhead. The function calls are resolved at compile time. It allows to achieve better performance than dynamic polymorphism.
- **Memory**: Static polymorphism requires no additional memory overhead. It does not require to store a pointer to the virtual-table in each object.
- **Value Semantics**: Static polymorphism allows to use value semantics. It allows to create objects on the stack and avoid dynamic memory allocation.
- **No Inheritance**: Static polymorphism does not require to use inheritance. It allows to create a more flexible design.

**Cons:**

- **Compile Time**: Static polymorphism requires to set the behavior of the object at compile time. It does not allow to change the behavior of the object at runtime.

### 1.3.3 Polymorphic Wrappers

`TODO: In the future`

## 1.4 Basic OOP Techniques

### 1.4.1 Inheritance

Inheritance is a mechanism in which one class acquires the properties and behavior of another class. It allows to define a new class based on an existing class.

One can distinguish between two types of inheritance:

- **Inheritance of implementation** (*code reuse*) and
- **Inheritance of interface** (*polymorphism*).

#### 1.4.1.1 Inheritance of Implementation

Inheritance of implementation is a technique that allows to reuse the code of an existing class. It allows to define a new class that inherits the attributes and methods (its implementation) of an existing class.

In C++ inheritance of implementation can be achieved using **private inheritance**.

Example:

```
class Set : private std::set<int> {
    using BaseImpl = std::set<int>;

public:
    using BaseImpl::BaseImpl;
```

```
    size_t size() const { return BaseImpl::size(); }

    const int& operator[](size_t index) const
    {
        return *std::next(BaseImpl::begin(), index);
    }

    bool add_item(int value)
    {
        return BaseImpl::insert(value).second;
    }

    bool remove_item(int value)
    {
        return BaseImpl::erase(value) > 0;
    }
};
```

#### 1.4.1.2 Inheritance of Interface

Defines when object of one type (*derived type*) can be used in place of an object of another type (*base type*).

In C++ inheritance of interface can be achieved using **public inheritance from a class that has only abstract methods**.

```
class Shape
{
public:
    virtual move(int dx, int dy) = 0
    virtual void draw() const = 0;
    virtual ~Shape() = default;
};

class Square : public Shape
{
    Rectangle rect_;
public:
    Square(int x, int y, int size) : rect_(x, y, size, size) {}
    void move(int dx, int dy) override { rect_.move(dx, dy); }
    void draw() const override { rect_.draw(); }
};
```

When the clients are using a reference or a pointer to the interface class they are unaware of the actual implementation of the object. This allows to change the implementation without affecting the client.

```
void draw_shapes(const std::vector<std::unique_ptr<Shape>>& shapes)
{
    for (const auto& shape : shapes)
    {
        shape->draw();
    }
}
```

#### 1.4.1.3 Inheritance - Pros and Cons

**Pros:**

- **Code Reusability**: Inheritance allows to reuse the code of an existing class.
- **Polymorphism**: Inheritance of interfaces allows to define a polymorphic scenario for a group of classes.

**Cons:**

- **Can Breaks Encapsulation**: Inheritance from classes with implementation creates a tight coupling between the base class and the derived class. It is especially problematic when the base class has **protected** data members. It reveals the internal implementation of the base class (breaks encapsulation) to the derived class. Any change in the base class can break the derived classes.
- **Static Behavior**: Inheritance is a static mechanism. The behavior of the derived class is determined at compile time. As a result it is not possible (or at least very difficult) to change the behavior of the derived class at runtime.

> **Important**
>
> ***Prefer interface inheritance over implementation inheritance!!!***

### 1.4.2 Composition

Composition is a technique in which one class contains an object of another class. It allows to create complex types by combining objects of other types.

Composition is a "has-a" relationship between two classes.

Example:

```
class RecentlyUsedList
{
    std::deque<std::string> list_;
public:
    RecentlyUsedList() = default;

    void add_item(std::string value)
    {
        list_.push_front(std::move(value));
    }

    const int& recent() const
    {
        return list_.front();
    }
};
```

#### 1.4.2.1 Composition - Pros and Cons

**Pros:**

- **Code Reusability**: Composition allows to reuse the code of an existing class.
- **Encapsulation**: Using composition one can only use the public interface of the composed class. It hides the internal implementation of the composed class.
- **Flexibility**: Composition is a dynamic mechanism. References or pointers to the composed objects can be set at run-time. It allows to set or change the behavior of the object at runtime.
- **Cohesion** Composition allows to create classes that have a single responsibility. It is easier to maintain and test such classes.

**Cons:**

- **Complexity**: Composition can lead to a more complex design. It requires to manage the lifetime of the composed objects.
- **Tight Coupling**: When composition uses concrete classes instead of the interfaces it can lead to a tight coupling between the composed objects. It can make the code harder to maintain.

> **Prefer composition over inheritance!!!**

### 1.4.3 Delegation

Delegation is a technique in which one object forwards a method call to another object. It allows to reuse the code of an existing class. It is a form of composition, however in static typed languages like C++ it is requires also usage of inheritance.

In delegation there are two objects involved in the process of handling a request:

- **Delegator**: The object that receives the request from a client. Instead of handling the request itself, it forwards the request to the delegate object.
- **Delegate**: The object that implements handling the request. It is the object that actually performs the request. It encapsulates and provides the behavior that is required by the delegator.

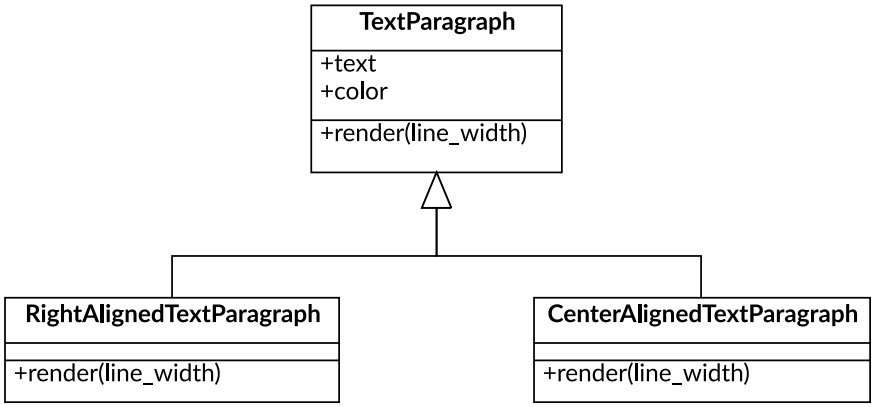
The following example compares the delegation and inheritance techniques.

- Code that uses inheritance:

```
class TextParagraph
{
    std::string text_;
    Color color_;

public:
    TextParagraph(std::string text, Color color) : text_(std::move(text)),
color_(std::move(color))
    {
    }

    const std::string& text() const {
        return text_;
    }

    virtual void render(size_t line_width) const
    {
        std::cout << "[" << text() << std::setw(line_width - text().length())
<< std::right << "" << "]\n";
    }

    virtual ~TextParagraph() noexcept = default;

    // other methods...
};

class RightAlignedTextParagraph : public TextParagraph
{
public:
    using TextParagraph::TextParagraph;

    void render(size_t line_width) const override
    {
        std::cout << "[" << std::setw(line_width) << std::right << text() <<
"]\n";
    }
};

class CenteredAlignedTextParagraph : public TextParagraph
{
public:
    using TextParagraph::TextParagraph;

    void render(size_t line_width) const override
    {
        auto pad_left = (line_width - text().length()) / 2;
        auto pad_right = line_width - text().length() - pad_left;
        std::cout << "[" << std::setw(pad_left) << "" << text() <<
```

```
std::setw(pad_right) << "" << "]\n" ;
    }
};
```

UML diagram for the code above:




- Code that uses delegation:

`TextAlignment` is an interface that defines the behavior of the classes that control the alignment of the text. The instance of `TextParagraph` gets the `render()` request but instead to implement the alignment by itself it delegates the request to the `TextAlignment` object.

```
class TextAlignment
{
public:
    virtual std::string aligned_text(const std::string& text, size_t
line_width) const = 0;
    virtual ~TextAlignment() = default;
};

class LeftAlignment : public TextAlignment
{
public:
    std::string aligned_text(const std::string& text, size_t line_width) const
override
    {
        std::stringstream out_str;
        out_str << text << std::setw(line_width - text.length()) << std::right
<< "";
        return out_str.str();
    }
};

class CenterAlignment : public TextAlignment
{
public:
    std::string aligned_text(const std::string& text, size_t line_width) const
override
    {
        std::stringstream out_str;
        auto pad_left = (line_width - text.length()) / 2;
        auto pad_right = line_width - text.length() - pad_left;
        out_str << std::setw(pad_left) << "" << text << std::setw(pad_right)
<< "";
        return out_str.str();
    }
};

class RightAlignment : public TextAlignment
```

```
{
public:
    std::string aligned_text(const std::string& text, size_t line_width) const
override
    {
        std::stringstream out_str;
        out_str << std::setw(line_width) << std::right << text;
        return out_str.str();
    };
};

class TextParagraph
{
    std::string text_;
    Color color_;
    std::unique_ptr<TextAlignment> alignment_;

public:
    TextParagraph(std::string text, Color color,
std::unique_ptr<TextAlignment> alignment = std::make_unique<LeftAlignment>())
        : text_(std::move(text))
        , color_(std::move(color))
        , alignment_{std::move(alignment)}
    {
    }

    void set_alignment(std::unique_ptr<TextAlignment> alignment)
    {
        alignment_ = std::move(alignment);
    }

    void render(size_t line_width) const
    {
        std::cout << "[" << alignment_->aligned_text(text_, line_width) <<
"]\n";
    }
};
```

Alignment of the text paragraph can be set and even changed at runtime:

```
TextParagraph text{"This is sample of text...", Color{0, 0, 0}};
text.render(80);

text.set_alignment(std::make_unique<RightAlignment>());
text.render(80);

text.set_alignment(std::make_unique<CenterAlignment>());
text.render(80);
```

#### 1.4.3.1 Delegation - Pros and Cons

**Pros:**

- **Code Reusability**: Delegation allows to reuse the code of an existing class.
- **Encapsulation**: Using delegation one can only use the public interface of the delegate class. It hides the internal implementation of the delegate class.
- **Loose Coupling**: Delegation allows to create a loose coupling between the delegator and the delegate objects. It allows to change the behavior of the delegator without affecting the delegate object.
- **Flexibility**: Delegation is a dynamic mechanism. References or pointers to the delegate objects can be set at run-time. It allows to set or change the behavior of the object at runtime.
- **Cohesion** Delegation allows to create classes that have a single responsibility. It is easier to maintain and test such classes.

**Cons:**

- **Complexity**: Delegation can lead to a more complex design. It requires to manage the lifetime of the delegate objects.

> **Delegation is more powerful way of introducing new behavior to the existing classes than inheritance!!!**

## 2. S.O.L.I.D. Principles

SOLID is an acronym representing five key design principles for Object-Oriented Programming (OOP). These principles were introduced by Robert C. Martin (also known as Uncle Bob) and aim to make software systems more maintainable, scalable, and flexible. Here's what SOLID stands for:

1. **Single Responsibility Principle (SRP)**: Every class should have one and only one reason to change. In essence, each class should handle only one responsibility to keep it focused and cohesive.
2. **Open/Closed Principle (OCP)**: Classes should be open for extension (adding new functionality) but closed for modification (no need to alter existing code). This makes systems easier to expand without introducing bugs into existing code.
3. **Liskov Substitution Principle (LSP)**: Subclasses should be able to replace their parent class without altering the correctness of the program. In other words, objects of a subclass must behave like objects of their superclass.
4. **Interface Segregation Principle (ISP)**: A class should not be forced to implement interfaces it does not use. Instead, break larger interfaces into smaller, more specific ones.
5. **Dependency Inversion Principle (DIP)**: High-level modules should not depend on low-level modules. Both should depend on abstractions, not concrete implementations. This reduces coupling and increases flexibility.

## 2.1 Single Responsibility Principle

> **Important**
>
> A class should have one, and only one, reason to change. This means that a class should have only one responsibility.

When a class has multiple responsibilities, any change in one responsibility might affect the others, leading to unexpected bugs or making the system fragile.

SRP promotes modularity and separation of concerns, allowing developers to make changes to one part of the code without fear of breaking other parts.

SRP ensures that a class focuses on a single purpose, which makes your code easier to:

- **Understand**: Since the class has a clear, specific role.
- **Maintain**: Changes related to that role don't affect unrelated functionality.
- **Test**: Simplifies testing, as you only need to verify one responsibility.

### 2.1.1 Cohesion of Implementation

Cohesion refers to how well the methods and properties of a class work together to achieve a single, well-defined purpose. A class with high cohesion contains only methods and attributes that are closely related to its specific responsibility, which aligns directly with the SRP.

#### 2.1.1.1 High Cohesion

- The component is focused and self-contained.
- Easier to understand, test, and debug.
- For example a `UserManager` class that only has CRUD methods that manage users in the database

```
class UserManager
{
public:
    void create_user(const User& user)
    {
        // Code to create a user
    }
    {
        // Code to create a user
    }

    void delete_user(const std::string& user_name)
    {
        // Code to delete a user
    }

    void update_user(const User& user)
    {
        // Code to update a user
    }
    {
        // Code to update a user's password
    }

    void get_user(const std::string& username)
    {
        // Code to retrieve user information
    }
};
```

#### 2.1.1.2 Low Cohesion

- The component tries to do too many unrelated things.

- Difficult to maintain because changes in one aspect may break others.
- For example, a `UserManager` class that handles user management, email sending, generating reports, and logging:

```
class UserManager
{
public:
    void create_user(const User& user)
    {
        // Code to create a user
    }

    void send_email(const std::string& email)
    {
        // Code to send an email
    }

    void log_activity(const std::string& activity)
    {
        // Code to log user activity
    }

    void generate_report()
    {
        // Code to generate a report about user activities
    }
};
```

### 2.1.2 Example of SRP

Imagine a class that handles both user authentication and logging:

```
class UserManager
{
public:
    void authenticate_user(const std::string& username, const std::string&
password)
    {
        // Code for user authentication
    }

    void log_activity(const std::string& activity)
    {
        // Code for logging user activity
    }
};
```

This class violates SRP because it has two responsibilities which can independently change:

- Authenticating users.
- Logging user activities.

We can fix this by separating the responsibilities into two classes:

```
class Authenticator
{
public:
    void authenticate_user(const std::string& username, const std::string&
password)
    {
        // Code for user authentication
    }
};

class Logger
{
public:
```

```
    void log_activity(const std::string& activity)
    {
        // Code for logging user activity
    }
};
```

Now, each class has a single responsibility:

- `Authenticator` handles user authentication.
- `Logger` handles logging activities.

### 2.1.3 Benefits of SRP

- **Encapsulation**: Keeps responsibilities isolated.
- **Flexibility**: You can modify or replace specific functionalities without impacting others.
- **Clean Code**: Makes your architecture more modular and easier to work with.

## 2.2 Open/Closed Principle

> **Important**
>
> Software entities (such as classes, modules, or functions) should be open for extension but closed for modification

What This Means:

- **"Open for extension"** means you should be able to add new functionality to a class or module without altering its existing code.
- **"Closed for modification"** means that once a class is written and tested, its behavior should not be changed.

If a class has specific behavior that works correctly, modifications to its functioning code should be avoided. To extend the code (add new functionality), a new class should be created using inheritance, where selected methods can be overridden and adapted to current requirements. While the class code cannot be modified (Closed Principle), it remains open to extension (Open Principle).

Introducing new functionality should not require changes to the code of the clients using it.

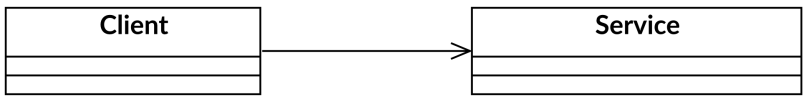
Classes open to extension should include **extension points** that allow new functionality to integrate seamlessly into the existing code. These extension points can be defined by **extracting an interface** for specific functionality.

### 2.2.1 Strong Coupling vs. Loose Coupling

**Strong coupling** refers to a situation where classes or modules are tightly interconnected, meaning that changes in one class can significantly impact others. This can lead to a fragile system where modifications in one part can cause unexpected behavior in another.

**Loose coupling**, on the other hand, refers to a design where classes or modules are independent of each other. Changes in one class have minimal or no impact on others. This is achieved through the use of interfaces, abstract classes, and dependency injection.

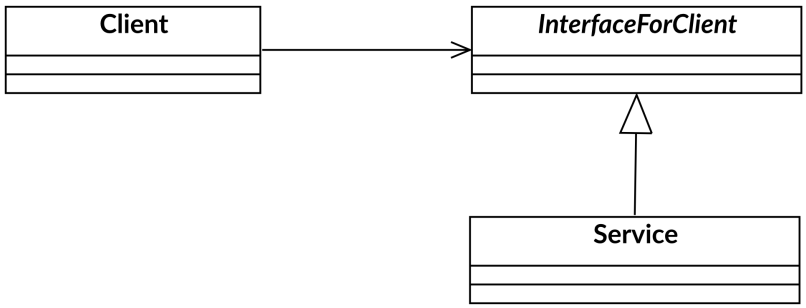
Let's consider an example:




In this case, the specific `Client` class uses a specific `Server` class. These classes are strongly coupled.

If we wanted the `Client` class object to use an object from a different server class, we would need to change the name of the server class being used within the `Client` class.

This project can be transformed to comply with the Open/Closed Principle (OCP) by applying the Strategy Design Pattern. We can extract an interface for the functionality called `InterfaceForClient`, which is used by the `Client` class.



Through this newly created extension point, we can substitute various implementations of the interface without needing to modify the client's code. Introducing new functionality in this setup involves writing a new class implementing the `InterfaceForClient`. The implementation of the `Client` class itself remains unchanged.

Now classes are loosely coupled, meaning that changes in one class do not affect others. This makes the system more flexible and easier to maintain.

The Open/Closed Principle (OCP) combines the concepts of encapsulation and interface extraction. It involves identifying consistent behaviors, extracting an interface, and using subclasses to handle new or changed behaviors.

### 2.2.2 Example of OCP

Let's consider a simple example of a `UserService` class that sends notifications to users. Initially, it only supports email and SMS notifications:

```
class UserService
{
    enum class NotificationType
    {
        Email,
        SMS
    };
public:
    void notify_user(const User& user)
    {
        if (notification_type_ == NotificationType::Email)
        {
            // sending email to a user
        }
        else if (notification_type_ == NotificationType::SMS)
        {
            // sending SMS to a user
        }
    }

    void set_notification_type(NotificationType type)
    {
        notification_type_ = type;
    }
private:
    NotificationType notification_type_ = NotificationType::Email;
};
```

In this example, the `UserService` class is responsible for sending notifications to users. However, if we want to add support for a new notification type (e.g., push notifications), we would need to modify the existing code, which violates the OCP.

To adhere to the OCP, we can refactor the code by introducing an interface for notifications:

```
class NotificationService
{
public:
    virtual ~NotificationService() = default;
    virtual void notify(const User& user) = 0;
};

class EmailNotificationService : public NotificationService
{
public:
    void notify(const User& user) override
    {
        // sending email to a user
    }
};
```

```
class SMSNotificationService : public NotificationService
{
public:
    void notify(const User& user) override
    {
        // sending SMS to a user
    }
};
```

Now, we can create a new class for push notifications without modifying the existing code:

```
class PushNotificationService : public NotificationService
{
public:
    void notify(const User& user) override
    {
        // sending push notification to a user
    }
};
```

Now, the `UserService` class can use any notification service without needing to know the details of how each service works:

```
class UserService
{
    NotificationService& notification_service_;
public:
    explicit UserService(NotificationService& service) noexcept :
notification_service_(service) {}

    void notify_user(const User& user)
    {
        notification_service_.notify(user);
    }
};
```

This example is a simple illustration of how to apply the Open/Closed Principle. By using interfaces and polymorphism, we can extend the functionality of our code without modifying existing classes. This makes our code more maintainable and flexible, allowing us to add new features with minimal impact on existing code.

It is also an example of **Strategy Design Pattern**, where the `UserService` class uses a strategy (notification service) to perform its task. The strategy can be changed at runtime by passing different implementations of the `NotificationService` interface.

### 2.2.3 Benefits of OCP

- **Fewer bugs**: Reduces the likelihood of introducing bugs when extending functionality.
- **Extendable**: Promotes flexibility and scalability.
- **Loose coupling**: Encourages the use of abstraction and polymorphism, which improves maintainability by reducing dependencies between components. This allows changes in one part of the system to have minimal impact on others, making the codebase easier to understand and modify.

## 2.3 Liskov Substitution Principle

> **Important**
>
> Objects in a program should be replaceable with instances of their subtypes without altering the correctness of the program.

If `B` is a subclass of `A`, you should be able to use an instance of `B` wherever an instance of `A` is expected, without the program breaking. In other words, derived classes must behave in a way that is consistent with the expectations set by their base classes.

### 2.3.1 Example of LSP

Let's consider a simple example with a `Rectangle` class and a `Square` class that inherits from it:

```
class Rectangle
{
    size_t width_;
    size_t height_;
public:
    Rectangle(size_t width, size_t height) noexcept : width_(width),
height_(height) {}
    virtual ~Rectangle() = default;

    virtual size_t width() const { return width_; }
    virtual size_t height() const { return height_; }

    virtual void set_width(size_t width) { width_ = width; }
    virtual void set_height(size_t height) { height_ = height; }
};

class Square : public Rectangle
{
public:
    explicit Square(size_t size) : Rectangle(size, size) {}

    void set_width(size_t width) override
    {
        Rectangle::set_width(width);
        Rectangle::set_height(width);
    }

    void set_height(size_t height) override
    {
        Rectangle::set_width(height);
        Rectangle::set_height(height);
    }
};
```

The problem with this design that `Rectangle` has in its interface methods to set independently the width and height. Inheriting from `Rectangle` forces `Square` to override these methods to maintain the invariant that a square has equal width and height. This violates the LSP because substituting `Square` for `Rectangle` can lead to unexpected behavior for the client.

```
void process_rectangle(Rectangle& rect)
{
    rect.set_width(5);
    rect.set_height(10);
    size_t area = rect.width() * rect.height();
    assert(area == 50); // This assertion will fail if TRectangle is Square
}

Rectangle rect(2, 3);
```

```
process_rectangle(rect);   // OK

Square square(2);
process_rectangle(square); // Fails, as the area will not be 50
```

### 2.3.2 Design by Contracts

**Design by Contract (DbC)**, is a software design methodology where classes and methods come with clearly defined *"contracts"*. These contracts define precise agreements between a method's caller and the method itself, specifying:

- **Preconditions**: Conditions that must be true before a method or function is executed.
- **Postconditions**: Conditions that must be true after the execution of the method.
- **Invariants**: Conditions that must always remain true throughout the lifecycle of an object.

The Liskov Substitution Principle (LSP) complements DbC by ensuring that subclasses adhere to their base class's contracts. Together, DbC and LSP promote robust, reusable, and correct code.

#### 2.3.2.1 LSP in the Context of Contracts

The LSP requires that a derived class remains substitutable for its base class. In the context of DbC, this means:

- **Preconditions**: A derived class must not strengthen preconditions (i.e., it cannot impose stricter requirements than the base class).
- **Postconditions**: A derived class must honor the postconditions of the base class and can strengthen them (i.e., guarantee stronger results if needed). The derived class cannot weaken the postconditions (promises) made by the base class.
- **Invariants**: The derived class must maintain the invariants defined by the base class.

### 2.3.3 Benefits of LSP

- Promotes robust inheritance hierarchies.
- Ensures code remains extensible and reusable.
- Helps avoid unexpected behavior or runtime errors.

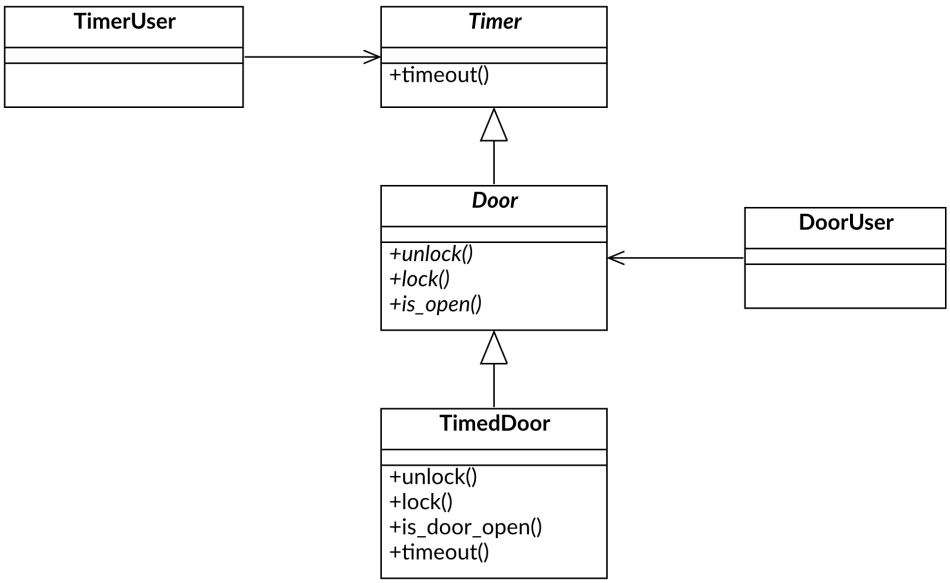
## 2.4 Interface Segregation Principle

> **Important**
>
> A client should never be forced to implement an interface that it doesn't use or clients shouldn't be forced to depend on methods they do not use.

Instead of creating large, monolithic interfaces with many methods, you should design smaller, more specific interfaces tailored to the exact needs of the clients (classes) that use them. This keeps the design clean, avoids unnecessary dependencies, and makes the code easier to maintain and extend.

### 2.4.1 Example of ISP

Let's consider a following diagram of classes:




In this example the user of the `Door` instance is forced to know about `Timer` interface and its `timeout()` method.

The better solution is to create a separate interface for the `Door` class that only includes the methods that are relevant to the user of the `Door` instance. Then we can use multiple inheritance to create `TimedDoor` class that implements both `Door` and `Timer` interfaces. But now we can pass the `TimedDoor` instance through the `Door` interface without needing to know about the `Timer` interface. The same object can be passed to the `TimerUser` using the `Timer` interface.

### 2.4.2 Benefits of ISP

- **Prevents Code Smells**: Large interfaces can lead to "fat interface" problems, where classes implement methods they don't actually need. This results in unused or meaningless code.
- **Improves Flexibility**: Smaller, focused interfaces are easier to implement and less likely to break existing code when modified.
- **Promotes Modularity**: Clear, specific interfaces lead to more modular and reusable components.

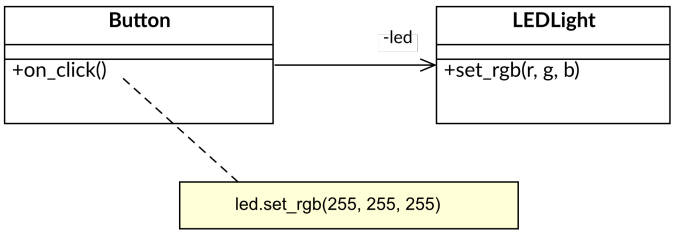
## 2.5 Dependency Inversion Principle

> **Important**
>
> 1. **High-level modules** should not depend on **low-level modules**. Both should depend on **abstractions** (e.g., interfaces).
> 2. **Abstractions** should not depend on details. Details (concrete implementations) should depend on abstractions.

Instead of high-level code (e.g., application logic) being tightly coupled to low-level code (e.g., specific implementations), both should rely on an abstraction (like an interface or abstract class). This helps reduce dependency between components and makes the system more adaptable to changes.

### 2.5.1 Example of DIP

Let's consider a simple example when using a button we want to turn on a LED indicator.




When `ToggleButton` uses the `LEDLight` class directly to turn on the light, it creates a tight coupling between the two classes. If we want to change the LED implementation (e.g., to a different type of light), we would need to modify the `ToggleButton` class.

This kind of design violates the Dependency Inversion Principle:

1. The high-level module (`ToggleButton`) depends on a low-level module (`LEDLight`). When we want to turn on the light we need to directly call `set_rgb(255, 255, 255)` method of the `LEDLight` class. We push the knowledge how the low-level module works into the high-level module (we need to know what is RGB color space in order to use the LED light properly).
2. If we are forced to use different LED light implementations, we need to modify the `ToggleButton` class. Thus changes in the low-level module require updates in the high-level module code.

To comply with the Dependency Inversion Principle, we can introduce an interface called `ISwitch` that defines the methods for turning on and off the light.

The `ISwitch` interface defines the methods `on()` and `off()` that the `ToggleButton` class can use without knowing the details of how the light works. This interface reflects what high-level module needs to operate. The `LEDSwitch` class implements this interface and provides the specific implementation for turning on and off the LED light. This is a part of a low-level module that depends on the `ISwitch` interface. Now both high-level and low-level modules depend on the abstraction (`ISwitch` interface), not on each other. The `LEDSwitch` works as an adapter that adapts the `LEDLight` class to the `ISwitch` interface created for a high-level module.

Future changes in the low-level module (e.g., changing the LED light implementation) will not affect the high-level module (`ToggleButton` class). The `ToggleButton` class can work with any implementation of the `ISwitch` interface, making it flexible and adaptable to changes.

### 2.5.2 Benefits of DIP

- **Encourages the use of abstractions**: Promotes flexibility and reusability by decoupling high-level and low-level modules.
- **Encourages loose coupling between components**: Makes your application easier to test and maintain.
- **Facilitates separation of concerns across architectural layers**: Aligns with best practices for clean architecture.

# 3. Design Patterns

> Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice.
>
> — Christopher Alexander, "A Pattern Language: Towns, Buildings, Construction"

> A description of communicating objects and classes that are customized to solve a general design problem in a particular context.
>
> — Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, "Design Patterns: Elements of Reusable Object-Oriented Software"

== Design Pattern

A Design Pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. They are designed to help you write code that is easier to understand, maintain, and extend.

#### 3.0.1 Key Aspects of Design Patterns

- **Reusable Solutions:**

Design patterns provide a proven solution that can be applied across different scenarios and projects.

- **Contextual:**

They are not one-size-fits-all but are applied to specific problems within a certain context, providing flexibility in implementation.

- **Best Practices:**

Patterns are developed from the experience of many developers over time, representing best practices for solving particular problems.

## 3.1 Elements of a Design Pattern

A design pattern has **four essential elements**:

1. **Pattern Name** – A shorthand that can be used to concisely describe the design problem, its solution, and consequences. It allows designing at a higher level of abstraction.
2. **Context/Problem** – Specifies when to apply the pattern.
3. **Solution** – Describes the elements that make up the solution to the defined problem, their relationships, responsibilities, and collaborations. It does not describe a specific design or implementation.
4. **Consequences** – The advantages and disadvantages of using the pattern.

A pattern provides an abstract description of a design problem and how a general arrangement of elements (classes and objects) solves that problem.

## 3.2 Pros & Cons of Design Patterns

#### 3.2.1 Pros of Design Patterns

1. **Reusability:**

Design patterns provide proven solutions to common problems, promoting code reuse and reducing redundancy.

1. **Efficiency:**

Using established patterns can speed up development as developers don't need to solve problems from scratch.

1. **Consistency**

Patterns ensure consistency in code structure, making it easier for multiple developers to work on the same codebase.

1. **Communication:**

Design patterns provide a **common vocabulary for developers**, simplifying discussions about software design and architecture.

1. **Maintainability:**

Well-defined patterns can make code easier to understand and maintain, as the structure and purpose of the code are clearer.

1. **Scalability:**

Patterns can provide scalable solutions that are flexible enough to handle growth and changes in requirements.

#### 3.2.2 Cons of Design Patterns

1. **Overhead:**

Implementing patterns can sometimes introduce additional layers of complexity, making the codebase harder to understand for newcomers.

1. **Overuse:**

There's a risk of overusing design patterns, applying them even when simpler solutions would suffice, leading to over-engineering.

1. **Learning Curve:**

Understanding and correctly implementing design patterns requires a certain level of expertise and experience, which can be a barrier for novice developers.

1. **Misapplication:**

Patterns can be misapplied to problems they weren't designed to solve, resulting in inefficient or convoluted code.

1. **Maintenance Challenges:**

While patterns can enhance maintainability, they can also make code harder to modify if the pattern isn't well understood or if the requirements change significantly.

1. **Performance:**

Some patterns can introduce performance overhead due to additional abstraction layers, which might not be suitable for high-performance applications.

## 3.3 Categories of Design Patterns

Design patterns are often categorized into three main types:

tablex(columns: 3, header-rows: 1, repeat-header: true, ..tableStyle, ..columnStyle,
[ **Creational Patterns** ], [ **Structural Patterns** ], [ **Behavioral Patterns** ], [ Factory Method ], [ Adapter ], [ Template Method ], [ Abstract Factory ], [ Decorator ], [ Strategy ], [ Prototype ], [ Composite ], [ State ], [ Bridge ], [ Proxy ], [ Observer ], [ Singleton ], [ Facade ], [ Chain of Responsibility ], [ Builder ], [ Bridge ], [ Command ], [ ], [ Flyweight ], [ Memento ], [ ], [ ], [ Mediator ], [ ], [ ], [ Visitor ], [ ], [ ], [ Interpreter ], [ ], [ ], [ Iterator ], )

# 4. Creational Patterns

Direct object creation is not always the best approach for instantiating objects. In such cases we have to deal with the complexity of object creation (organize dependencies and pass them to the constructor). This makes the code strongly coupled with its dependencies. Creational design patterns provide an abstract way to create objects while hiding the creation logic, rather than instantiating objects directly using constructor.

The main goal of creational design patterns is to provide better alternatives for situations where a direct object creation is not convenient. These patterns provide an abstraction layer that hides the actual object creation process. This can help improve the flexibility of the code.

## 4.1 Pros of Creational Patterns

1. **Encapsulation:**

Creational patterns encapsulate the object creation process, making it easier to change the object creation process without affecting the client code.

1. **Flexibility:**

Creational patterns provide a way to create objects in a controlled manner, allowing you to change the object creation process at runtime.

1. **Reusability:**

Creational patterns promote code reuse by providing a generic way to create objects.

1. **Decoupling:**

Creational patterns help decouple the client code from the object creation process, making it easier to maintain and test the code.

## 4.2 Factory Method

### 4.2.1 Intent

**Factory Method** is a creational design pattern that provides an interface for creating objects and allows its implementors to alter the type of objects that will be created.

### 4.2.2 Example - from problem to solution

The class is strongly coupled to the concrete classes it creates. This makes it hard to change the class's behavior or replace the concrete classes with other classes.

```
class MusicApp
{
public:
    //...

    void play(const std::string& track_title)
    {
        // creation of the object
        SpotifyService music_service("spotify_user", "rjdaslf276%2", 45);

        // usage of the object
        std::optional<Track> track = music_service.get_track(track_title);
        if (track.has_value())
        {
            // ... play the track
        }
        else
        {
            //... handle error
        }
    }

    //...rest of the class
};
```

The **Factory Method** pattern suggests defining an interface for creating objects, but lets subclasses decide which class to instantiate. This way, the class delegates the object creation to its subclasses.

We can start with extracting an interface for the music service:

```
class MusicService
{
public:
    virtual std::optional<Track> get_track(const std::string& title) = 0;
    virtual void play(Track track) = 0;
    virtual ~MusicService() = default;
};
```

Any specific music service should implement this interface. For example, the `SpotifyService` or `AppleMusicService`.

```
class SpotifyService : public MusicService
{
public:
    SpotifyService(const std::string& user, const std::string& password, int
port);
    std::optional<Track> get_track(const std::string& title) override;
    void play(Track track) override;
};
```

Then we can abstract the process of creating the music service instance:

```
class MusicServiceCreator
{
```

```
public:
    virtual std::unique_ptr<MusicService> create_music_service() = 0;
    virtual ~MusicServiceCreator() = default;
};
```

The `MusicApp` class can now delegate the responsibility of creating the music service to the objects implementing the `MusicServiceCreator` interface. The creator can be injected as dependency to the `MusicApp` class.

```
class MusicApp
{
    std::shared_ptr<MusicServiceCreator> music_service_creator_;
public:
    MusicApp(std::shared_ptr<MusicServiceCreator> music_service_creator) // DI
- Dependency Injection
        : music_service_creator_(music_service_creator)
    {
    }

    void play(const std::string& track_title)
    {
        // creation of the object
        std::unique_ptr<MusicService> music_service =
            music_service_creator_->create_music_service();

        // usage of the object
        std::optional<Track> track = music_service->get_track(track_title);

        //...
    }
};
```

Now the process of creating the music service is encapsulated in the `MusicServiceCreator` object. The `MusicApp` class is no longer responsible for creating the music service object. Moreover, the `MusicApp` class is no longer coupled to the concrete class of the music service. It can work with any class that implements the `MusicServiceCreator` interface. The process of instantiating the music service is separated from the usage of the music service.

### 4.2.3 Factory Method - Context

We want to introduce a new functionality by writing a new class and creating an instance of that class.

### 4.2.4 Factory Method - Problem

- We want to instantiate concrete classes through an abstract interface
- A given class cannot anticipate the type of object to be created
- Information about the type of object to be created is available only at runtime
- Object creation should be separated from the object usage

### 4.2.5 Factory Method - Structure

### 4.2.6 Factory Method - Collaboration

- The `Creator` class object delegates to its derived classes the responsibility of defining the factory method to create an instance of the appropriate `ConcreteProduct` class (a subclass of the abstract `Product` class)

### 4.2.7 Factory Method - Consequences

- Eliminates the need to insert specific classes into the application code.
- Creating objects within a class using a factory method is more flexible than creating them directly.
- Promotes loose coupling by reducing dependencies to concrete classes specific classes in your code.

### 4.2.8 Parallel Class Hierarchies

- Parallel class hierarchies arise when a class delegates some of its responsibilities to a separate class

- The Factory Method pattern allows combining parallel class hierarchies

```
std::shared_ptr<Shape> selected_shape = get_clicked_shape();

std::shared_ptr<Manipulator> manipulator = selected_shape-
>create_manipulator();

manipulator->on_drag(100, 200);
```

#### 4.2.8.1 Factory Method - Implementation

- We can implement the factory class as:
  - An abstract class (interface) that does not provide implementations for the methods it declares
  - A concrete class that provides a default implementation of the factory method, but can be overridden by derived classes
- In a simple case, the factory interface can be replaced with a type

```
using MusicServiceCreator = std::function<std::unique_ptr<MusicService>()>;
```

#### 4.2.8.2 Parameterized Factories

Parameterized factories are used when the type of object to be created depends on the identifier passed to the factory method. As a result, a single factory instance can create objects of different types. Identifiers can be of different types, such as `std::string`, `enum`, or any other type that can be used to determine the type of object to be created. The information what type of object to create arrives very often at runtime (when parsing configuration files, user input, etc.).

Such a factory can be implemented as follows:

```
using MusicServiceCreator = std::function<std::unique_ptr<MusicService>()>;

class MusicServiceFactory
{
    std::unordered_map<std::string, MusicServiceCreator> creators_;
public:
    std::unique_ptr<MusicService> create(const std::string& id) const
    {
        auto& creator = creators_.at(id);
        return creator();
    }

    bool register_creator(const std::string& id, const MusicServiceCreator&
creator)
    {
        return creators_.emplace(id, creator).second;
    }
};
```

The creators are registered with the factory using the `register_creator` method. As a creator, we can use a lambda function that creates an instance of the desired class.

```
MusicServiceFactory music_service_factory;

music_service_factory.register_creator("Tidal",
    [] { return std::make_unique<TidalService>("tidal_user",
"KJH8324d&df"); });

//...
```

```
std::string id_from_config = "Tidal";
MusicApp app(music_service_factory.create(id_from_config));
```

#### 4.2.8.3 Factory Method - Related Patterns

- Iterator
- Abstract Factory – an abstract factory is often implemented using factory methods
- Template Method – factory methods are often called within an algorithm implemented as template method

## 4.3 Abstract Factory

### 4.3.1 Intent

The **Abstract Factory** is a creational design pattern that provides an interface for creating families of related or dependent objects without specifying their concrete classes.

### 4.3.2 Example - from problem to solution

Let's write a framework that allows us to use different database engines. We can start with introducing a set of interfaces that represent:

- a connection to the database
- a command that can be executed on the database
- a transaction that can be started and committed

```
class DbConnection
{
public:
    virtual ~DbConnection() = default;
    virtual void connect() = 0;
    virtual void disconnect() = 0;
};

class DbCommand
{
public:
    virtual ~DbCommand() = default;
    virtual DbResult execute(DbConnection& conn, const SqlCmd& cmd, const
SqlParams& params) = 0;
};

class DbTransaction
{
public:
    virtual ~DbTransaction() = default;
    virtual void begin(DbConnection& conn) = 0;
    virtual void commit() = 0;
};
```

In order to handle specific database engines, we can create a set of concrete classes that implement the above interfaces. For example, we can create a `OracleConnection`, `OracleCommand`, and `OracleTransaction` for Oracle database engine. The same applies to other database engines like PostgreSQL or SQLite.

```
// Family of related products for Oracle database engine
class OracleConnection : public DbConnection
{
public:
    void connect() override
    {
        // connect to Oracle database
    }

    void disconnect() override
    {
        // disconnect from Oracle database
    }
};

class OracleCommand : public DbCommand
{
public:
    DbResult execute(DbConnection& conn, const SqlCmd& cmd, const SqlParams&
params) override
    {
```

```
        // execute command on Oracle database
    }
};

class OracleTransaction : public DbTransaction
{
public:
    void begin(DbConnection& conn) override
    {
        // begin transaction on Oracle database
    }

    void commit() override
    {
        // commit transaction on Oracle database
    }
};
```

Classes that handle Sql Server or PostgreSQL can be implemented in a similar way. The important part is that all of them implement the same interfaces, which allows us to use them interchangeably.

Having the different families of products we still need a way to create them. We don't want to expose the concrete classes to the client code. This would lead to a situation where the client code is tightly coupled to the concrete classes, making it hard to change the implementation later. For example, if we want to switch from Oracle to PostgreSQL, we would need to change all the places in the code where we create `OracleConnection`, `OracleCommand`, and `OracleTransaction` objects. This is not a good design.

```
void insert_to_db()
{
    auto connection = std::make_unique<OracleConnection>("localhost", "db",
"user", "password", 5432);
    connection->connect();

    auto transaction = std::make_unique<OracleTransaction>();
    transaction->begin(*connection);

    auto command = std::make_unique<OracleCommand>();
    command->execute(*command, "INSERT INTO users (name, age) VALUES (?, ?)",
{ "John", 30 });

    transaction->commit();
}
```

Instead, we want to provide a way to create the objects without specifying their concrete classes. This is where the **Abstract Factory** comes into play. We can create an interface that defines methods for creating the different families of products.

```
class DbFactory
{
public:
    virtual ~DbFactory() = default;
    virtual std::unique_ptr<DbConnection> create_connection(const std::string&
url, const std::string& db_name, const std::string& user, const std::string&
password) = 0;
    virtual std::unique_ptr<DbCommand> create_command() = 0;
    virtual std::unique_ptr<DbTransaction> create_transaction() = 0;
};
```

We can then create concrete factories for each database engine. For example, we can create `OracleFactory`, `PostgreSqlFactory`, and `SqlServerFactory`.

```
class OracleFactory : public DbFactory
{
```

```
public:
    std::unique_ptr<DbConnection> create_connection(const std::string& url,
const std::string& db_name, const std::string& user, const std::string&
password) override
    {
        return std::make_unique<OracleConnection>(url, db_name, user,
password);
    }

    std::unique_ptr<DbCommand> create_command() override
    {
        return std::make_unique<OracleCommand>();
    }

    std::unique_ptr<DbTransaction> create_transaction() override
    {
        return std::make_unique<OracleTransaction>();
    }
};
```

Now we can use `DbFactory` interface to create the objects without specifying their concrete classes. This allows us to switch between different database engines easily.

```
void insert_to_db(DbFactory& factory)
{
    auto connection = factory.create_connection("localhost", "db", "user",
"password");
    connection->connect();

    auto transaction = factory.create_transaction();
    transaction->begin(*connection);

    auto command = factory.create_command();
    command->execute(*command, "INSERT INTO users (name, age) VALUES (?, ?)",
{ "John", 30 });

    transaction->commit();
}

int main()
{
    OracleFactory oracle_factory;
    insert_to_db(oracle_factory);

    PostgreSqlFactory postgresql_factory;
    insert_to_db(postgresql_factory);

    SqlServerFactory sqlserver_factory;
    insert_to_db(sqlserver_factory);
}
```

In this example, we can easily switch between different database engines by passing a different factory to the `insert_to_db` function. This allows us to create the objects without specifying their concrete classes, which is the main goal of the **Abstract Factory** pattern.

### 4.3.3 Abstract Factory - Context

We have a families of related objects that differ in their implementation. For example, we have a family of database engines (Oracle, PostgreSQL, SQL Server) and we want to create a set of objects that represent the connection, command, and transaction for each database engine.

### 4.3.4 Abstract Factory - Problem

- We want to create a family of related objects without specifying their concrete classes
- We want to be able to switch between different families of products easily

- We want to avoid tight coupling between the client code and the concrete classes
- We should be able to add new families of products without changing the existing code

#### 4.3.5 Abstract Factory - Solution

- Create an interface that defines methods for creating the different families of products
- Create concrete factories for each family of products that implement the interface
- Use the factory interface to create the objects without specifying their concrete classes

#### 4.3.6 Abstract Factory - Consequences

- The client code is decoupled from the concrete classes, which makes it easier to switch between different families of products
- The client code is easier to maintain and extend, as we can add new families of products without changing the existing code
- Adding a new product to a family is difficult, as we need to modify the factory interface and all the concrete factories

#### 4.3.7 Abstract Factory vs. Factory Method

- The **Factory Method** pattern provides an interface for creating a single object without specifying its concrete class. It is used when we want to create a single object that can be of different types, but we don't want to expose the concrete classes to the client code.
- The **Abstract Factory** pattern provides an interface for creating families of related or dependent objects without specifying their concrete classes. It is used when we want to easily replace a set of related objects that are designed to work together, ensuring consistency among the objects created by the factory.
- Typical solution for the **Abstract Factory** pattern is to use an interface that consists of several factory methods - each for a product in the family.

## 4.4 Prototype

### 4.4.1 Intent

The **Prototype** pattern is a creational design pattern that allows cloning objects, even complex ones, without coupling to their specific classes. It's useful when you want to create a new object by copying an existing object, and you don't know the exact type of the object you want to copy.

### 4.4.2 Example - from problem to solution

Let's say we are developing a drawing application that allows users to create and manipulate shapes. The application should support different types of shapes, such as circles, squares, and triangles. Each shape has its own properties and methods for drawing and manipulating it. The hierarchy of shapes is represented by the following classes:

```
class Shape
{
public:
    virtual ~Shape() = default;
    virtual void draw() = 0;
    virtual void move(int dx, int dy) = 0;
};

class Circle : public Shape
{
public:
    Circle(int x, int y, int radius) : x_(x), y_(y), radius_(radius) {}
    void draw() override { /* draw circle */ }
    void move(int dx, int dy) override { x_ += dx; y_ += dy; }
private:
    int x_, y_, radius_;
};

class Square : public Shape
{
public:
    Square(int x, int y, int side) : x_(x), y_(y), side_(side) {}
    void draw() override { /* draw square */ }
    void move(int dx, int dy) override { x_ += dx; y_ += dy; }
private:
    int x_, y_, side_;
};

// etc.
```

We would like to implement a Copy/Paste feature in our application, allowing users to duplicate shapes. However, the current design requires us to know the exact type of shape we want to copy, which can be cumbersome and error-prone. A typical implementation of the Copy/Paste feature might look like this:

```
void copy_selected_shape()
{
    std::unique_ptr<Shape> copied_shape = nullptr;

    Shape* selected_shape = get_selected_shape();

    if (selected_shape)
    {
        if (Circle* circle = dynamic_cast<Circle*>(selected_shape); circle)
        {
            copied_shape = std::make_unique<Circle>(*circle); // call of copy
constructor for Circle
        }
        else if (Square* square = dynamic_cast<Square*>(selected_shape);
square)
```

```
        {
            copied_shape = std::make_unique<Square>(*square); // call of copy
constructor for Square
        }
        // etc.

        if (copied_shape)
        {
            add_to_clipboard(copied_shape);
        }
    }
}
```

The problem with this approach is that it requires us to know the exact type of shape we want to copy. If we add a new shape type, we need to modify the `copy_selected_shape()` function to handle the new type. This violates the Open/Closed Principle (OCP), which states that software entities should be open for extension but closed for modification.

What we need is a way to create a new shape by copying an existing one without knowing its exact type. This is where the **Prototype** pattern comes in.

The solution is to add a `clone()` method to the `Shape` class and implement it in each derived class. The `clone()` method can be used by the clients to create a copy of the shape without knowing its exact type:

```
class Shape
{
public:
    virtual ~Shape() = default;
    virtual void draw() = 0;
    virtual void move(int dx, int dy) = 0;
    virtual std::unique_ptr<Shape> clone() const = 0; // Prototype method
};
```

The `clone()` method has to be implemented in each derived class:

```
class Circle : public Shape
{
public:
    Circle(int x, int y, int radius) : x_(x), y_(y), radius_(radius) {}
    void draw() override { /* draw circle */ }
    void move(int dx, int dy) override { x_ += dx; y_ += dy; }

    std::unique_ptr<Shape> clone() const override
    {
        return std::make_unique<Circle>(*this); // call of copy constructor
for Circle
    }
};

class Square : public Shape
{
public:
    Square(int x, int y, int side) : x_(x), y_(y), side_(side) {}
    void draw() override { /* draw square */ }
    void move(int dx, int dy) override { x_ += dx; y_ += dy; }

    std::unique_ptr<Shape> clone() const override
    {
        return std::make_unique<Square>(*this); // call of copy constructor
for Square
    }
};
```

Now we can implement the `copy_selected_shape()` function without knowing the exact type of shape we want to copy:

```
void copy_selected_shape()
{
    std::unique_ptr<Shape> copied_shape = nullptr;

    Shape* selected_shape = get_selected_shape();

    if (selected_shape)
    {
        copied_shape = selected_shape->clone(); // call of clone method
        add_to_clipboard(copied_shape);
    }
}
```

The code is now cleaner and adheres to the OCP. If we add a new shape type, we only need to implement the `clone()` method in that class, and the `copy_selected_shape()` function will work without any modifications.

### 4.4.3 CRTP for cloning

Now introduction of a new shape requires overriding the `clone()` method in the derived class. It's implementation is very similar in each subclass:

```
class Rectangle : public Shape
{
public:
    std::unique_ptr<Shape> clone() const override
    {
        return std::make_unique<Rectangle>(*this); // call of copy constructor
for Rectangle
    }
    //.. rest of the class
};
```

The only thing we have to change is a name of a concreate type passed to `std::make_unique` function. We can use the **Curiously Recurring Template Pattern** (CRTP) to avoid code duplication in the `clone()` method. Let us introduce a `CloneableShape` class template that will implement the `clone()` method using CRTP:

```
template <typename TShape>
class CloneableShape : public Shape
{
public:
    std::unique_ptr<Shape> clone() const override
    {
        // Use static_cast to ensure the correct type is passed to the copy
constructor.
        // This converts the current object (*this) to the derived type
(TShape) safely.
        return std::make_unique<TShape>(static_cast<const TShape&>(*this));
    }
};
```

Static cast is used to convert `*this` reference (which is `const CloneableShape<TShape>&`) to the reference type that will allow calling the copy constructor for an object of the type passed as a template parameter.

The advantage of CRTP is that it eliminates repetitive code by providing a reusable implementation of the `clone()` method. This ensures consistency and reduces the likelihood of errors when adding new shape types. For example, a new shape type (`Rectangle`) can inherit from `CloneableShape<Rectangle>`, which automatically injects the correct implementation of the `clone()` method:

```
class Rectangle : public CloneableShape<Rectangle>
{
public:
    Rectangle(int x, int y, int width, int height) : x_(x), y_(y),
```

```
width_(width), height_(height) {}
    void draw() override { /* draw rectangle */ }
    void move(int dx, int dy) override { x_ += dx; y_ += dy; }
};
```

### 4.4.4 Prototype and Liskov Substitution Principle (LSP)

The **Prototype** pattern relies on the **Liskov Substitution Principle** (LSP), which states that objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program. In the context of the Prototype pattern, this means that the `clone()` method should return a pointer to the base class (`Shape`) but create an object of the derived class (e.g., `Circle`, `Square`, etc.). This allows us to treat all shapes uniformly while still being able to create specific instances of each shape type.

Unfortunately, we can easily introduce a bug when we inherit from a concrete class and forget to override the `clone()` method. To adhere to the Liskov Substitution Principle (LSP) and ensure correct behavior, the `clone()` method must be explicitly overridden in every derived class. If not, the derived class will not be able to clone itself correctly, which can lead to unexpected behavior:

```
class ColorCircle : public Circle
{
public:
    ColorCircle(int x, int y, int radius, const Color& color) : Circle(x, y,
radius), color_(color) {}
    void draw() override { /* draw colored circle */ }
    void move(int dx, int dy) override { Circle::move(dx, dy); }
private:
    Color color_;
};

void liskov_substitution_violation(Shape& shape)
{
    std::unique_ptr<Shape> cloned_shape = shape.clone(); // client should get
the exact type of the shape
    cloned_shape->draw();
}

int main()
{
    auto color_circle = std::make_unique<ColorCircle>(0, 0, 10, Color::Red);
    liskov_substitution_violation(*color_circle); // This will call the draw
method of Circle instead of ColorCircle
}
```

This is a violation of the LSP because the derived class (`ColorCircle`) does not behave as expected when used in place of the base class (`Shape`). The `clone()` method should be implemented in each concrete class that derives from `Shape` to ensure that it returns a pointer to the correct type. This bug is very difficult to find, because the code compiles and runs. But at some point (after clonning) it runs in a wrong state.

## 4.5 Builder

The **Builder** pattern is a creational design pattern that allows constructing complex objects step by step. It separates the construction of a complex object from its representation, allowing the same construction process to create different representations.

## 4.6 Singleton

The **Singleton** pattern is a creational design pattern that ensures a class has only one instance and provides a global point of access to that instance.